Exactness Properties of LCN Marginal-Inference Algorithms

A consolidated reference: what each engine bounds, when it is exact, outer, inner, or unreliable — with formal results and proofs

IBM Research

Abstract

The LCN library ships nine marginal-inference engines — the exact oracle `ExactInference`, the junction-tree NLP `CredalJT` (scheme D5), the credal-network family Credal VE, Credal CTE, ApproxLP, Interval BP, Credal IJGP, and the structural engines ARIEL and SCC-factor-graph propagation. Their exactness behavior has been analyzed one engine at a time in six companion notes (`ariel_exactness`, `cve_exactness`, `ccte_exactness`, `ibp_exactness`, `approxlp_exactness`, `tighter_approximation`). This document consolidates that analysis into a single reference. We fix one framework — the LCN distribution set $\mathcal{D}_{\mathrm{LCN}}$, its strong extension $\mathcal{D}_{\mathrm{SE}} \supseteq \mathcal{D}_{\mathrm{LCN}}$, and the two looseness sources (Gap A, Gap B) — and then place every engine in it: which feasible set it optimizes, whether its returned interval is *exact*, a valid *outer* bound, an *inner* bound (hence *not* a valid outer bound), or *unreliable* (a solver artifact); the precise topological condition under which it is exact; and its cost. We state the formal results with proof sketches, collected in one place: the fundamental inclusion $\mathcal{D}_{\mathrm{LCN}} \subseteq \mathcal{D}_{\mathrm{SE}}$; CVE's exactness over $\mathcal{D}_{\mathrm{SE}}$; the exactness of CVE/IBP/ARIEL on clean singly-connected LCNs; ApproxLP's inner-bound guarantee; IBP's mode dichotomy; IJGP's exactness at i-bound $\geq$ treewidth; ARIEL's structural LMC preservation; and the D5/ CredalJT exactness theorem (hypotheses H1–H4, with the projection/reconstruction proof). Throughout we separate three distinct kinds of looseness — *definitional* (an engine that honestly targets $\mathcal{D}_{\mathrm{SE}}$), *solver* (ipopt local optima in `ExactInference`), and *implementation* (the now-fixed ARIEL factor-grouping bug; the two now-fixed Interval BP message bugs) — and report the *shipped* reality, not merely the intended theory. A master table and a topology cross-cut table give the one-screen summary, and a closing decision guide says which engine to reach for.

Contents

1 The unifying framework

1.1 LCNs, the LCN distribution set, and the marginal task

An LCN over binary atoms $\mathbf{V} = \{V_1, \ldots, V_n\}$ is a set of probability *sentences*, each Type-1 $\ell \leq P(\varphi) \leq u$ or Type-2 $\ell \leq P(\varphi \mid \psi) \leq u$ over propositional formulas, together with the conditional independencies of the *Local Markov Condition* (LMC) of its primal graph. These jointly define the *LCN distribution set*

$$\mathcal{D}_{\mathrm{LCN}} = \big\{\, p \in \Delta(2^{\mathbf{V}}) \;:\; g_s(p) \in [\ell_s, u_s] \;\forall \text{sentences } s, \quad h_\iota(p) = 0 \;\forall \text{LMC assertions } \iota \,\big\}, \tag{1}$$

where each g_s is a Type-1 linear or Type-2 ratio functional and each h_ι is the bilinear LMC equality $P(x, y, s)\,P(s) = P(x, s)\,P(y, s)$ for the assertion $(X \perp\!\!\!\perp Y \mid S)$, over every joint configuration of Y and S. The marginal task for a query atom a is

$$\underline{P}(a) = \min_{p \in \mathcal{D}_{\mathrm{LCN}}} P(a), \qquad \overline{P}(a) = \max_{p \in \mathcal{D}_{\mathrm{LCN}}} P(a),$$

and, for a conditional query, the ratio $P(a \mid e)$. Every functional in (1) depends on p only through its marginal on a fixed atom set (*locality*): sentence s through $p{\restriction}\mathrm{atoms}(s)$, assertion $(X \perp\!\!\!\perp Y \mid S)$ through $p{\restriction}(X \cup Y \cup S)$. Locality is the lever behind every exactness result below.

1.2 The credal network and the strong extension

When the LCN's structure graph is a chain graph, the joint factorizes over families:

$$P(\mathbf{V}) = \prod_v P(\mathrm{ch}_v \mid \mathrm{pa}_v), \tag{2}$$

and this factorization reproduces exactly the LMC independencies. The `cn/` pipeline (`CredalNetworkVertices.from_lcn`) compiles the LCN into a *credal network*: interval-valued local credal sets K_v with enumerated extreme points. The credal-network engines bound not $\mathcal{D}_{\mathrm{LCN}}$ but the *strong extension*

$$\mathcal{D}_{\mathrm{SE}} = \Big\{ \textstyle\prod_v P_v \;:\; P_v \in K_v \text{ chosen independently per parent-config} \Big\}. \tag{3}$$

Proposition 1 (Fundamental inclusion)**.** $\mathcal{D}_{\mathrm{LCN}} \subseteq \mathcal{D}_{\mathrm{SE}}$. *The inclusion is equality when the primal graph is singly connected and there is no cross-family sentence (a sentence whose atom scope lies in no single family); it is strict exactly when a cross-family sentence couples choices that the free product of* (3) *decouples.*

Proof sketch. Any $p \in \mathcal{D}_{\mathrm{LCN}}$ factorizes as (2) and each conditional $P(\mathrm{ch}_v \mid \mathrm{pa}_v)$ lies in K_v, so $p \in \mathcal{D}_{\mathrm{SE}}$: the inclusion holds. A chain-graph product satisfies every LMC assertion by construction, so a free product in (3) can leave $\mathcal{D}_{\mathrm{LCN}}$ *only* by violating a *sentence*; when every sentence is intra-family, each K_v already encodes every sentence constraining its scope and the free product of compliant families is compliant, giving $\mathcal{D}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$. A cross-family sentence, by contrast, can be violated by a vertex combination that each family satisfies locally — e.g. on `d4_biting` the free product reaches $P(B \wedge C) \approx 0.9$ against the sentence bound 0.05 — so $\mathcal{D}_{\mathrm{SE}} \supsetneq \mathcal{D}_{\mathrm{LCN}}$. □

Consequently any engine that returns a valid bound over $\mathcal{D}_{\mathrm{SE}}$ is automatically a valid *outer* bound over $\mathcal{D}_{\mathrm{LCN}}$: $[\underline{q}_{\mathrm{LCN}}, \overline{q}_{\mathrm{LCN}}] \subseteq [\underline{q}_{\mathrm{SE}}, \overline{q}_{\mathrm{SE}}]$. The gap between the two closes precisely under the Proposition 1 condition.

1.3 The two looseness sources

Following `tighter_approximation.tex`, the gap between a credal-network engine's output and the LCN truth has two independent origins.

- **Gap A (local-set widening).** Each K_v is computed per family in isolation, so it admits conditionals no global $p \in \mathcal{D}_{\mathrm{LCN}}$ realizes: it ignores sentences on *other* families and the cross-family LMC equalities. Formally $K_v^{\mathrm{true}} \subseteq K_v^{\mathrm{local}}$, often strictly (e.g. the vacuous $[0, 1]$ CPT rows on `alarm`).
- **Gap B (strong-extension widening).** Even with $K_v = K_v^{\mathrm{true}}$, the free product of (3) lets each family pick its vertex independently, violating cross-family sentences. This is exactly the strict inclusion of Proposition 1.

The tightening schemes of `tighter_approximation.tex` target these: D1 (`method="linear-tight"`), D2 (`merge_budget`), D3 (`solver="scip"`) attack Gap A at build time and are free for every credal-network engine; D4 (`coupling="cross-family"`) and D5 (`CredalJT`) attack Gap B at propagation time.

1.4 The bound-character taxonomy

We classify each engine's returned interval $[\underline{q}, \overline{q}]$ against the truth $[\underline{q}^\star, \overline{q}^\star]$ (either $\mathcal{D}_{\mathrm{LCN}}$ or $\mathcal{D}_{\mathrm{SE}}$, stated per engine).

Definition 1 (Bound character)**.** *The interval is* exact *if* $[\underline{q}, \overline{q}] = [\underline{q}^\star, \overline{q}^\star]$*; a valid* outer *bound if* $[\underline{q}, \overline{q}] \supseteq [\underline{q}^\star, \overline{q}^\star]$ *(sound: it never excludes a feasible marginal); an* inner *bound if* $[\underline{q}, \overline{q}] \subseteq [\underline{q}^\star, \overline{q}^\star]$ *(*not *a valid outer bound — it can be too tight); and* unreliable *if neither containment is guaranteed (a solver or implementation artifact).*

The crucial practical point: an *inner* bound is unsafe as a guarantee — it may claim more certainty than the LCN supports. ApproxLP and Interval BP's variational mode are inner *by design*; the (now-fixed) ARIEL factor-grouping bug and Interval BP message bugs used to make those engines inner *unsoundly.*

1.5 Three kinds of looseness

It is essential to keep apart *why* an engine's interval is not the LCN truth:

1. **Definitional** — the engine honestly targets $\mathcal{D}_{\mathrm{SE}}$, and $\mathcal{D}_{\mathrm{SE}} \supsetneq \mathcal{D}_{\mathrm{LCN}}$ (Gap B) or $K_v^{\mathrm{local}} \supsetneq K_v^{\mathrm{true}}$ (Gap A). This is sound looseness: the interval is a valid outer bound, just not tight. Closed by D1–D5.

2. **Solver** — the engine targets the right set but uses a local optimizer. This is `ExactInference`'s `solver="local"` (ipopt on a nonconvex bilinear NLP): a local optimum can report a *spuriously inner* (too-tight, unsound) interval. Cured by `solver="global"` (SCIP).

3. **Implementation** — a bug makes the shipped engine depart from its own algorithm. Two were on record and *both are now fixed*: the ARIEL factor-grouping bug (which dropped the largest factor's sentences, §5.1) and the two Interval BP message bugs (§4.4). We report the *shipped* status, not just the intended theory; with both fixes landed the shipped engines now match their algorithms.

2 Master table

Table 1 is the one-screen summary. "Targets" is the feasible set the engine optimizes; "character" is per Definition 1 against that set; "exact (LCN) when" is the topological condition for the returned interval to equal the $\mathcal{D}_{\mathrm{LCN}}$ truth.

Table 1: Exactness properties of the LCN marginal-inference engines. In the "exact when" column, "clean s.c." abbreviates $\mathcal{D}_{\mathrm{SE}}$=$\mathcal{D}_{\mathrm{LCN}}$, i.e. a singly-connected primal graph with no cross-family sentence.

Engine	Targets	Character	Exact (LCN) when	Cost	Shipped
`ExactInference` (global)	$\mathcal{D}_{\mathrm{LCN}}$	exact*	always (gap $\leq$ tol)	2^n; SCIP	OK
`ExactInference` (local)	$\mathcal{D}_{\mathrm{LCN}}$	unreliable	— (may over-tighten)	2^n; ipopt	caveat
`CredalJT` (D5)	$\mathcal{D}_{\mathrm{LCN}}$	outer; exact if H1–H4	H1–H4 (Thm. 1)	treewidth	OK
Credal VE	$\mathcal{D}_{\mathrm{SE}}$	exact over $\mathcal{D}_{\mathrm{SE}}$	clean s.c.	per-target elim	OK
Credal CTE (ϵ=0)	$\mathcal{D}_{\mathrm{SE}}$	exact over $\mathcal{D}_{\mathrm{SE}}$†	clean s.c.	two-pass tree	OK†
ApproxLP	$\mathcal{D}_{\mathrm{SE}}$	*inner*	clean s.c., no stall	coord. descent	OK
Interval BP (`interval`)	$\mathcal{D}_{\mathrm{SE}}$	outer	clean s.c. (tree)	loopy messages	fixed
Interval BP (`variational`)	$\mathcal{D}_{\mathrm{SE}}$	*inner*	(inner sample)	mean-field	fixed
Credal IJGP	$\mathcal{D}_{\mathrm{SE}}$	outer-style	clean s.c., $i \geq$ tw	2^i/cluster	OK
ARIEL	$\mathcal{D}_{\mathrm{LCN}}$	outer; exact on s.c.	singly connected	factor messages	fixed
SCCP / SCCP-ARIEL	$\mathcal{D}_{\mathrm{LCN}}$	outer	trivial SCCs	per-SCC NLP	caveat

* certified when the SCIP gap $\leq$ `gap_tol`; on time-out the bound is valid but its optimality is unconfirmed. † Credal CTE is exact over $\mathcal{D}_{\mathrm{SE}}$ on a clean singly-connected sentence-free LCN, but its single shared elimination order falls *strictly inner* the moment a cross-family sentence appears (§4.2). "Shipped" flags the implementation status: *fixed* (ARIEL, §5.1; Interval BP, §4.4 — each had a bug, since corrected), *caveat* (`ExactInference`-local, SCCP).

3 The exact family: targeting $\mathcal{D}_{\mathrm{LCN}}$

3.1 `ExactInference`: the definitional oracle

`ExactInference` (`exact.py`) solves, for each atom, a min and a max of $P(a)$ (or the ratio $P(a \mid e)$) over the full 2^n joint subject to *all* sentences and *all* LMC equalities — i.e. directly over $\mathcal{D}_{\mathrm{LCN}}$ of (1). It is the definitional target, not a member of the credal-network family; in particular it does *not* bound $\mathcal{D}_{\mathrm{SE}}$. Two backends, selected by the `solver` argument of `run`:

- `solver="global"`: SCIP spatial branch-and-bound. Certifies the global optimum, or on time-out returns the best bound with its optimality gap. `self.status` records the per-atom verdict (`confirmed` / `unconfirmed` / `unsolved`).
- `solver="local"` (default): ipopt with a two-phase SLSQP fallback (`find_feasible_points` → `optimize_marginal_slsqp`) and multi-restart. Fast, but a *local* method on a nonconvex bilinear-LMC NLP.

Proposition 2 (Global certified; local unreliable)**.** *With `solver="global"` and reported gap ≤ `gap_tol`, `ExactInference` returns the exact $\mathcal{D}_{\mathrm{LCN}}$ bounds. With `solver="local"`, the returned interval is* unreliable *(Definition 1): it may be spuriously inner (too tight), because a local optimum of the nonconvex NLP can sit strictly inside the true range.*

Proof sketch. The model (1) is exactly $\mathcal{D}_{\mathrm{LCN}}$, so any *globally* optimal min / max equals the true bound; SCIP's spatial B&B certifies global optimality up to the gap tolerance. ipopt, being a local NLP solver on a nonconvex feasible region (the bilinear LMC equalities are nonconvex), can converge to a stationary point whose objective is strictly interior to the true extremum; the multi-restart + SLSQP fallback mitigates but does not certify. Hence the local backend carries no soundness guarantee. □

Witness (shipped). On `alarm.lcn` the certified bound is $P(A) = [0.087, 0.100]$ (brute-force / SCIP), whereas raw `solver="local"` reports the spuriously tighter $P(A) = [0.092, 0.100]$, $P(E) = [0.050, 0.073]$ — an *inner*, unsound interval. This is the reason the companion notes treat `ExactInference`-local as *not* a reliable oracle and prefer SCIP-backed cross-checks (`verify_scip.py`, `verify_bruteforce.py`); `CredalJT` is often the more trustworthy engine on such instances (§3.2).

3.2 `CredalJT` (D5): exact at treewidth cost

`CredalJT` (`cn/junction_nlp.py`) builds one junction (clique) tree from the chain-graph elimination order, gives each cluster C a variable vector q_C over the joint of its atoms, ties adjacent clusters by Lauritzen separator-consistency equalities, and hosts every sentence and LMC equality on a cluster containing its atoms. It reads each query marginal off a host cluster and optimizes min / max with SCIP. Cost is $\sum_C 2^{|\mathrm{atoms}(C)|}$ — exponential in *treewidth*, not in n. Let $\mathcal{F}$ denote $\mathcal{D}_{\mathrm{LCN}}$ of (1) and

$$\mathcal{G} = \big\{(q_C)_{C\in T} : \textstyle\sum_i q_C[i] = 1; \quad M_{C\to S} q_C = M_{D\to S} q_D \text{ on every edge } (C,D),\ S{=}C\cap D; \\ \text{each hosted constraint holds}\big\}$$

be the cluster-feasible region. An LMC assertion $(X \perp\!\!\!\perp Y \mid S)$ is *hosted* if some cluster contains $X \cup Y \cup S$, and *separator-entailed* if S separates X from Y in the chordal graph G^* whose maximal cliques are the clusters.

Theorem 1 (D5 exactness)**.** *Suppose* (H1) *T has the running-intersection property (RIP; always true by construction);* (H2) *every LCN sentence is hosted;* (H3) *every LMC assertion is either hosted or separator-entailed in G^*;* (H4) *the query atom and all evidence atoms lie together in some cluster. Then for every such atom a,*

$$\min_{q\in\mathcal{G}}(\textit{read } a) = \min_{p\in\mathcal{F}} p_a, \qquad \max_{q\in\mathcal{G}}(\textit{read } a) = \max_{p\in\mathcal{F}} p_a,$$

and likewise for the conditional ratio $P(a \mid e)$. That is, `CredalJT` *equals* `ExactInference`*.*

Proof sketch. Projection $\mathcal{F} \to \mathcal{G}$ (always valid; needs only H1/H4). Given $p \in \mathcal{F}$, set $q_C := p{\restriction}C$. Cluster simplexes hold; each separator equality holds because both sides equal $p{\restriction}S$; every hosted constraint holds by locality; the read-off of a equals p_a. Hence $q \in \mathcal{G}$ with the same objective, so $[\min,\max]_{\mathcal{G}} \supseteq [\min,\max]_{\mathcal{F}}$: **CredalJT is always a valid outer bound.**

Reconstruction $\mathcal{G} \to \mathcal{F}$ (needs H2/H3). Given a separator-consistent family $(q_C) \in \mathcal{G}$, the RIP marginal-consistency theorem (Lauritzen) yields a unique joint $p = \prod_C q_C / \prod_{(C,D)\in T} q_S$ whose cluster marginals are the q_C. Verify $p \in \mathcal{F}$: each *hosted* sentence/LMC holds because, by locality, its value on p equals its value on the host cluster marginal, which satisfies it (H2, hosted part of H3). Each *separator-entailed* LMC holds because p factorizes over T and hence obeys the global Markov property of G^*: whenever S separates X from Y in G^*, $X \perp\!\!\!\perp_p Y \mid S$ holds automatically, even though the assertion was dropped from $\mathcal{G}$ (other part of H3). Thus $[\min,\max]_{\mathcal{F}} \supseteq [\min,\max]_{\mathcal{G}}$. Both inclusions force equality; H4 makes the conditional ratio a functional of a single cluster, so the same two maps preserve it. □

Three consequences. (a) The proof never mentions chains: exactness is a property of the *junction tree*, not of chains. (b) The projection half is unconditional, so `CredalJT` is *never unsound* — its interval always contains the truth; H2/H3 decide only tightness. (c) On a Markov chain the moral graph is already triangulated and every LMC is single-separator, so H3 holds with treewidth 1. *H3 is sufficient but not necessary*: on `asia` and `polytree` some LMCs are neither hosted nor separator-entailed yet CredalJT is empirically exact (the unentailed assertions do not bind); on `smokers` the six large-scope LMCs of the undirected clique $F_1F_2F_3$ are unhostable and unentailed, H3 fails *and bites*, and CredalJT returns the loose but valid $P(F_1) = [0, 1]$.

4 The credal-network family: targeting $\mathcal{D}_{\mathrm{SE}}$

All five engines here consume the same `CredalNetworkVertices`; they differ only in how they propagate the enumerated extreme points, and hence in where they land relative to $[\underline{q}_{\mathrm{SE}}, \overline{q}_{\mathrm{SE}}]$.

4.1 Credal VE: exact over the strong extension

Credal VE (`cve.py`) computes every atom's bounds by looping a single-query bucket elimination, each time recomputing an elimination order in which the target is eliminated *last*.

Proposition 3 (CVE exact over $\mathcal{D}_{\mathrm{SE}}$)**.** *With* `coupling="off"` *and exact pruning (ϵ=0), Credal VE returns exactly $[\underline{q}_{\mathrm{SE}}(a), \overline{q}_{\mathrm{SE}}(a)]$ for every atom, hence a valid outer bound on $\mathcal{D}_{\mathrm{LCN}}$.*

Proof sketch. Every potential CVE forms is a finite set of products of local extreme points, i.e. a set of $\mathcal{D}_{\mathrm{SE}}$ vertices; combine and marginalize preserve this, and pruning removes only dominated functions, which never attain a componentwise extremum. Because $\mathcal{D}_{\mathrm{SE}}$ is a product of finitely many polytopes, each bound is attained at a vertex combination; eliminating the query *last* keeps that combination in the surviving potential through to the read-off, so the reported min / max equals the $\mathcal{D}_{\mathrm{SE}}$ extremum. See `cve_exactness.tex` §3. □

Combined with Proposition 1, Credal VE is *exact w.r.t.* $\mathcal{D}_{\mathrm{LCN}}$ exactly when $\mathcal{D}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$ (singly connected, no cross-family sentence). It is the "gold" member of the family: exact over $\mathcal{D}_{\mathrm{SE}}$, whereas the others are inner (ApproxLP), outer with loop slack (IBP loopy), or order-sensitive (CTE).

4.2 Credal CTE: exact over $\mathcal{D}_{\mathrm{SE}}$, but order-sensitive

Credal CTE (`ccte.py`) is the two-pass (collect + distribute) bucket-tree engine: one shared elimination order produces all marginals at once. With ϵ=0 it is exact over $\mathcal{D}_{\mathrm{SE}}$ on a clean singly-connected sentence-free LCN (the shared schedule is a genuine clique tree there). But its single shared order — deliberately *not* augmented per query, unlike Credal VE — can prune the function carrying an extremal marginal at an intermediate scope where it is dominated. The witness is `d4_biting`: the true $\mathcal{D}_{\mathrm{SE}}$-and-$\mathcal{D}_{\mathrm{LCN}}$ value is $P(B) = [0.04, 0.72]$ (Credal VE, CredalJT, and two oracles agree), yet Credal CTE reports the strictly *inner* $[0.05, 0.65]$ (and CTE+D4 $[0.05, 0.55]$). So on cross-family instances CTE can fall inside $[\underline{q}_{\mathrm{SE}}, \overline{q}_{\mathrm{SE}}]$ — *not even a valid outer bound.* See `ccte_exactness.tex`.

4.3 ApproxLP: an inner bound

ApproxLP (`approxlp.py`) reformulates the marginal problem as a multilinear program and solves it by coordinate descent: fix all families but one, pick that family's best extreme point (a linear subproblem), sweep to convergence.

Proposition 4 (ApproxLP inner)**.** *With `coupling="off"`, ApproxLP returns an interval contained in $[\underline{q}_{\mathrm{SE}}, \overline{q}_{\mathrm{SE}}]$; in general it is* not *a valid outer bound on $\mathcal{D}_{\mathrm{LCN}}$.*

Proof sketch. Every state ApproxLP evaluates is a single feasible product joint (each family at one extreme point, or the center), i.e. a point of $\mathcal{D}_{\mathrm{SE}}$, and the objective is evaluated exactly on that precise Bayes net. The reported min/max is therefore a min/max over a *subset* of $\mathcal{D}_{\mathrm{SE}}$ (the coordinate-descent trajectory), hence no smaller / no larger than the global $\mathcal{D}_{\mathrm{SE}}$ extremum: $\underline{q}_{\mathrm{SE}} \le \underline{q}_{\mathrm{ALP}} \le \overline{q}_{\mathrm{ALP}} \le \overline{q}_{\mathrm{SE}}$. See `approxlp_exactness.tex` §2. □

The inner gap is strict only when the nonconvex program makes descent stall at a non-global local optimum. On the instances tested (`d4_biting`, 2-atom chain) it is *sound and tight*: it reproduces the exact $P(B) = [0.04, 0.72]$. Its natural use is the *inner* half of a two-sided bracket: ApproxLP (inner) with Credal VE (exact over $\mathcal{D}_{\mathrm{SE}}$) on the same vertices sandwich the truth and detect stalls.

4.4 Interval BP: outer / inner by mode; exact on trees

Interval BP (`ibp.py`, class `IntervalBP`) is loopy interval message passing over the factor graph. `method="interval"` passes per-state intervals and is an *outer* approximation of $[\underline{q}_{\mathrm{SE}}, \overline{q}_{\mathrm{SE}}]$; `method="variational"` runs mean-field over a finite sample of vertex combinations and is an *inner* approximation.

Proposition 5 (IBP interval outer; exact on tree)**.** *With `method="interval"`, Interval BP returns a valid outer bound over $\mathcal{D}_{\mathrm{SE}}$. When the factor graph is a tree (Markov chain, tree, polytree) it is exact over $\mathcal{D}_{\mathrm{SE}}$; combined with Proposition 1 it is then exact over $\mathcal{D}_{\mathrm{LCN}}$.*

Proof sketch. Each factor→child update ranges over all local extreme points and all corner *distributions* of the incoming messages, so every $\mathcal{D}_{\mathrm{SE}}$ marginal is contained in the message interval; a conditional factor sends the vacuous $[0, 1]$ upward, excluding no parent marginal; variable→factor messages only intersect. On a tree there is a unique path between factors, so no message feeds back and no correlated pair's corners over-cover, giving the exact $\mathcal{D}_{\mathrm{SE}}$ range. See `ibp_exactness.tex`. □

Shipped status: fixed. Two message bugs previously made the interval mode inner and unsound — a spurious normalized-likelihood upward message (collapsing an unconstrained root to $[0.5, 0.5]$) and an invalid component-wise corner enumeration. Both are now corrected; the shipped engine reproduces Credal VE on the 2-atom chain ($P(x_0) = [0.30, 0.50]$, was $[0.5, 0.5]$) and on `d4_biting` ($P(B) = [0.04, 0.72]$, was $[0.05, 0.65]$). One residual limitation is architectural: the intersection message algebra performs no Bayesian *upward* update, so evidence on a child does not shift a parent's posterior — for conditional queries with evidence on descendants use Credal VE / `CredalJT`.

4.5 Credal IJGP: exact at i-bound $\geq$ treewidth

Credal IJGP (`ijgp.py`) runs iterative join-graph propagation with a mini-bucket `i_bound` capping cluster size, forward+backward passes to a message-change threshold.

Proposition 6 (IJGP exact at large i-bound)**.** *When `i_bound` $\geq$ the treewidth of the credal network, the join graph is a genuine junction tree and Credal IJGP is exact over $\mathcal{D}_{\mathrm{SE}}$; with $\mathcal{D}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$ it is exact over $\mathcal{D}_{\mathrm{LCN}}$. For smaller `i_bound` it is an outer-style approximation, tunable by the i-bound.*

Proof sketch. At `i_bound` $\geq$ treewidth no mini-bucket partitioning is forced, so the join graph carries no cycle and message passing is exact elimination over the enumerated vertices (the same argument as Interval BP on a tree, one level up at cluster granularity). Below treewidth, mini-bucket splitting introduces the loopy slack that the i-bound trades against cost 2^i/cluster. □

5 Structural and hybrid engines: targeting $\mathcal{D}_{\mathrm{LCN}}$ by decomposition

5.1 ARIEL: structural LMC preservation

ARIEL (`ariel.py`) runs interval message passing on the bipartite factor graph whose factor nodes group sentences by shared atom scope.

Proposition 7 (ARIEL exact on singly-connected)**.** *On a singly-connected LCN, ARIEL returns the exact $\mathcal{D}_{\mathrm{LCN}}$ marginal bounds. For every LMC assertion ($X \perp\!\!\!\perp Y \mid S$): if $X \cup Y \cup S$ fits in one factor, the bilinear equality is imposed in that factor's local NLP; otherwise the assertion factorizes into single-separator independencies along the unique factor path, each realized exactly by an interval message across that separator. The interval summary is lossless precisely because a tree has no second path along which a discarded joint correlation could matter.*

Proof sketch. Structural LMC preservation on a tree factor graph (`ariel_exactness.tex` Prop. 1). On the shipped chain/tree/polytree, *no* LMC fits inside a single factor, so none is imposed as an explicit constraint, yet the assertions survive structurally along the tree paths, giving exactness. On a loopy factor graph the interval message across a boundary variable loses the joint correlation that a second path needs, so ARIEL is a valid outer bound but not tight (the coupled-diamond example, $P(D) = [0.316, 0.684]$ vs exact $[0.316, 0.594]$). □

Shipped status: fixed. An earlier factor-graph construction (`factor_graph.py`, `FactorGraph._make_factor_nodes`) had an off-by-one labelling bug: it created the first factor before the grouping loop and then reused the label `f1` for the next new-scope factor, silently overwriting — and dropping the sentences of — the first, largest-scope factor (sentences are sorted by decreasing scope). The *shipped* ARIEL therefore returned vacuous $[0, 1]$ interior marginals on chain/tree/polytree, contradicting Proposition 7. The bug is now fixed (each new-scope factor gets a fresh post-incremented label). Verified: on the two-atom chain the fixed engine returns $P(x_0) = [0.30, 0.50]$, $P(x_1) = [0.40, 0.68]$, matching Credal VE / `ExactInference`; all tests pass. See `ariel_exactness.tex`.

5.2 SCCP / SCCP-ARIEL: exactness on the acyclic condensation

The SCC-factor-graph engines (`sccp.py`, `sccp_ariel.py`) contract each strongly connected component of the structure graph into a super-node, giving an acyclic condensation DAG, and run two-pass (collect + distribute) belief propagation on it. On the condensation the propagation is exact elimination; the only inexactness is *inside* a cyclic super-node, whose local nonconvex NLP is solved by ipopt (`sccp.py`, local ⇒ the same reliability caveat as `ExactInference`-local) or approximately by ARIEL loopy BP (`sccp_ariel.py`). Hence: when every SCC is trivial (a singleton, whose local problem is an LP), the two-pass propagation is exact over $\mathcal{D}_{\mathrm{LCN}}$; a non-trivial SCC introduces solver looseness (v1) or approximation (v2). See `scc_factor_graph.tex`.

6 Topology cross-cut

Table 2 reads the master table along the other axis: for each canonical topology, the character of each engine's shipped output. "ex" = exact, "out" = valid outer bound (loose), "in" = inner (unsound as a guarantee), "unrel" = solver-unreliable.

Table 2: Bound character by topology (shipped engines; unconditional marginals).

Topology	Exact-glob	CredalJT	CVE	CTE	ApproxLP	IBP-int	IJGP[‡]
Markov chain (`chain`)	ex	ex	ex	ex	ex	ex	ex
Tree (`tree`)	ex	ex	ex	ex	ex	ex	ex
Polytree (`polytree`)	ex	ex[†]	ex	ex	ex	ex	ex
Chain graph, compound (`alarm`)	ex[*]	ex	ex	ex	ex	ex	ex
Cross-family sentence (`d4_biting`)	ex	ex	ex	**in**	ex	ex	out
Loopy DAG	ex	out	out[§]	out[§]	in[§]	out	out
Undirected clique (`smokers`)	ex	**out**	out[§]	out[§]	in[§]	out	out

[*] certified-global is exact; `ExactInference`-*local* over-tightens here ($P(A) = [0.092, 0.100]$ vs $[0.087, 0.100]$) — *unrel*. [†] H3 fails but CredalJT is empirically exact (unentailed LMCs do not bind). [‡] IJGP shown at a small i-bound; at i-bound $\geq$ treewidth it matches CredalJT. [§] On a loopy/undirected-clique LCN $\mathcal{D}_{\mathrm{SE}} \supsetneq \mathcal{D}_{\mathrm{LCN}}$; the $\mathcal{D}_{\mathrm{SE}}$-family entries are exact-over-$\mathcal{D}_{\mathrm{SE}}$ but only *outer* (CVE/CTE/IBP/IJGP) or *inner* (ApproxLP) over $\mathcal{D}_{\mathrm{LCN}}$. On `smokers` even CredalJT is outer-only ($P(F_1) = [0, 1]$). For cross-family / loopy instances the reliable exact route is `ExactInference`-global or `CredalJT` where H1–H4 hold.

7 Reliability and verification

Three verifiers cross-check the engines against ground truth (`lcn/inference/marginal/`):

- `verify_scip.py` — certified global (SCIP spatial B&B) marginal bounds; the reference oracle when it terminates within the gap tolerance.
- `verify_bruteforce.py` — LMC-aware brute-force enumeration cross-check; found the `ExactInference`-local over-tightening on `alarm`.
- `verify_lmc_factorization.py` — solver-free check that the credal network's product joints satisfy the LMC (certifies $\mathcal{D}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$ on chain/tree/polytree, i.e. Gap B$= 0$).

The reliability lessons carried into Table 1: `ExactInference`-local is a *local* solver (use `solver="global"` when a guarantee is needed); the ARIEL factor-grouping bug and the two Interval BP message bugs are all *fixed*, so those shipped engines now match their algorithms; Credal CTE is order-sensitive and can be inner on cross-family instances. When two engines that both claim "exact over $\mathcal{D}_{\mathrm{SE}}$" (CVE and CTE) disagree, the disagreement localizes an order-sensitivity or a bug, not a modeling difference.

8 Decision guide

- **Need a certified guarantee, small n:** `ExactInference` with `solver="global"`. Never trust `solver="local"` as a guarantee.
- **Need exactness at scale, bounded treewidth:** `CredalJT` (D5) — exact under H1–H4 at cost $\sum_C 2^{|C|}$, always a valid outer bound otherwise. This is the recommended exact engine for chain-graph LCNs, and is often more trustworthy than `ExactInference`-local (`alarm`).
- **All singleton marginals over $\mathcal{D}_{\mathrm{SE}}$, exactly:** Credal VE. Prefer over Credal CTE when a cross-family sentence is present (CTE can fall inner).
- **A two-sided sandwich of the truth:** pair Credal VE (outer over $\mathcal{D}_{\mathrm{SE}}$) with ApproxLP (inner over $\mathcal{D}_{\mathrm{SE}}$) on the same vertices; equal endpoints certify the $\mathcal{D}_{\mathrm{SE}}$ bound.
- **Fast, singly-connected:** Interval BP `method="interval"` or ARIEL — both exact on trees now that their respective bugs are fixed.
- **Tighten any $\mathcal{D}_{\mathrm{SE}}$-family engine toward $\mathcal{D}_{\mathrm{LCN}}$:** apply the build-stage schemes D1 (`method="linear-tight"`), D2 (`merge_budget`), D3 (`solver="scip"`) first — free for all of them — then D4/D5 for cross-family coupling.

The six companion notes hold the per-engine detail: `cve_exactness.tex`, `ccte_exactness.tex`, `ibp_exactness.tex`, `approxlp_exactness.tex`, `ariel_exactness.tex`, and `tighter_approximation.tex` (the D1–D5 taxonomy and the full D5 theorem).